6章: Python基礎 — バックエンドの言語（完全）

> （バックエンド） Python（パイソン）
> `04`
>
> アプリの実際のバックエンドコード
> 1行ずつ、 のまま
>
> `kosu/` `.py` 書いてあることが
> める

この章で学ぶこと

- Pythonとは何か
- インデント（字下げ）
- 数とデータ型
- リスト・辞書・タプル
- 条件 岐 `if` / 繰り し `for`
- リスト 記
- 関数 `def`
- クラスと `self` / `__init__` / `__str__` / 継承
- 外処 `try-except`
- `with`
- `import` と相 import
- 三項演算子・f 字 ・デコレータ `@`
- 実コード完全解説 `main_views.py` `main_utils.py` `asynchronous_views.py` `signals.py` `models.py`

次

init　str

0. 前　　識：Pythonはどこで　くのか

> Python（パイソン：プログラミング言語の　前）とは？　　みやすさ

0.1 フロントエンドとバックエンドの違い

	く場所	言語	割	アプリのフォルダ
フロントエンド				`frontend/`
バックエンド		Python/Django		`kosu/` `hozen_another/`

> たとえ　：レストラン　　客席とメニュー　　厨
>
> この章で学ぶPythonは、その厨　の言

0.2　アプリのPythonファイルの場所

`kosu/`

```
kosu/
├── models.py            # データの「設計図」（テーブルの形）。§17で解説
├── signals.py           # データ変更を自動で記録する仕掛け。§18で解説
├── urls.py              # どのURLでどの処理を呼ぶか（07章）
├── views/               # 各機能の処理本体
│   ├── main_views.py    # ログイン・メニュー・ログ。§14で解説
│   ├── asynchronous_views.py  # バックアップ等の重い処理。§16で解説
│   └── ...
└── utils/
    └── main_utils.py    # 便利な共通部品（検証関数など）。§15で解説
```

> 拡　子 `.py`　　ただの　字（テキスト）

0.3 Pythonの実行のされ　（おおまかに）

インタプリタ（interpreter：　）型
1行ずつその場で　しながら

```
あなたが書いた .py ファイル（人間の言葉に近い）
        ↓
Python本体（python.exe）が1行ずつ読んで
        ↓
コンピュータが分かる命令に翻訳しながら実行
```

> なぜ？
>
> 書いてすぐ試せる　`python`
> `manage.py runserver`　`01`　`07`

---

1. インデント — Python最　の

絶　に最初に押さえるべき

1.1 インデントとは

> インデント（indent：字下げ）とは？　空　（スペース）

カッコ `{ }`　`04`

```
// ※説明用の簡易例（JavaScript）
if (x > 0) {
  // { } の中が「if が成立したときの処理」
  console.log("正の数");
}
```

Python には　カッコがありません。　　インデント（字下げ）の深さ

> ▼ このコードがやること（先に　　語で）：

```
# ※説明用の簡易例（Python）
# 行末に ： （コロン）を付ける
if x > 0:
    # 字下げされた行が「if が成立したときの処理」
    print("正の数")
```

- `if x > 0:`　　行　に `:`（コロン）
- 　　`print(...)`　先　が字下げ

1.2 字下げの深さで「　・外」が決まる

> ▼ このコードがやること:

```
# ※説明用の簡易例
if x > 0:
    # 字下げあり → if の中（x>0 のときだけ実行）
    print("A")
    # 字下げあり → これも if の中
    print("B")
# 字下げなし → if の外（いつでも実行）
print("C")
```

- `A`　`B`　　if の　`x > 0`
- `C`　　if の外　`x`

> ⚠️ インデントは「　える」こと。混ぜると　エラー:　　角スペース
> 　　スペース2個
> 　　1つのファイルの　で　一されていれば

> スペースとタブ（Tabキー）を混ぜると `IndentationError`

> なぜPythonはわざわざこんなルールにしたの？

1.3　　のコードで確

`main_views.py`

```
# クラス定義（一番外側、字下げ0）
class PrintToLogger:
  # メソッド定義（字下げ1段＝クラスの中）
  def write(self, message):
    # if文（字下げ2段＝メソッドの中）
    if message.strip():
      # if の中身（字下げ3段）
      views_logger.info(message)
```

- `class PrintToLogger:`
- `def write(...):`
- `if message.strip():`
- `views_logger.info(message)`

この「コロン `:` で始まり、次の行から字下げで　身を書く」が、Pythonの全構　（if・for・def・class・try・with…）に共　する基　形

2.　数とデータ型

2.1　数 —　に　前を　ける

> 数（へんすう：　を　れておく　前　きの　）とは？

`let`　`const`　　宣言キーワードが　　`名前 = 値`

> ▼ このコードがやること:

```
# ※説明用の簡易例
# input_number という箱に 12345 を入れる
input_number = 12345
# name という箱に "admin"（文字）を入れる
name = "admin"
# authority という箱に True（真）を入れる
authority = True
```

- `=`　　「右の　を左の　に　れる（　）」
  `=`
- `==`

> 語：　（だいにゅう）　　`x = 5`

`main_views.py`

```
# data という箱に、解析したデータを入れる
data = json.loads(request.body)
# input_number に、その中の従業員番号を入れる
input_number = data.get('employee_no')
```

2.2 データ型 —　の種

種　（型）

型（よみ）	意		アプリでの　場
`int`		`12345` `-3` `0`	
`float`		`3.14` `0.5`	
`str`		`"admin"` `'P'`	
`bool`		`True` `False`	
`None`		`None`	

> 語：　字　（もじれつ／string）　　シングルクォート `'...'` でもダブルクォート `"..."` でも

> 語：真偽　（しんぎち／boolean）
>
> `True`（真）　`False`（偽）　　先　が　字
>
> `true`

> 語：`None`（ノン）
>
> `null`　　`None`

2.3 型の自　　と　換

自　で

```
# ※説明用の簡易例
# 5 は整数なので x は int 型
x = 5
# "5" は文字なので x は str 型（同じxでも型が変わる）
x = "5"
```

> ⚠️ 「5」と「"5"」は　：`5`　　`"5"`　　まったく　の型　　`5 + 3`　`8`　　`"5" + "3"`　　`"53"`
>
> `main_utils.py`　　`int(value)`　数　に　換

```
# ※説明用の簡易例
# 文字 "123" → 数値 123
int("123")
# 数値 123 → 文字 "123"
str(123)
```

---

3. リスト・辞書・タプル

　　1個ずつ　　まとめて

3.1 リスト（list）— 順　きの　び

> リスト（list：　数の　を順　に　べた　れ　）とは？　`[ ]`

> ▼ このコードがやること:

```
# ※説明用の簡易例
# 角カッコで囲んだリスト
numbers = [10, 20, 30]
# → 10 （0番目。番号は0から始まる！）
print(numbers[0])
# → 20 （1番目）
print(numbers[1])
# → [10, 20, 30, 40] 末尾に追加
numbers.append(40)
```

- `[ ]`
- `numbers[0]` 0 0から数えます
- `.append(値)`

`main_views.py`

```
# 空のリストを作る
follow_message_list = []
...
# メッセージを末尾に追加していく
follow_message_list.append(follow_message)
```

> `[]` `.append()`

3.2 辞書（dict）— 前と のペア

> 辞書（dict：キーと をペアで持つ れ ）とは？ `{ }` キー（ 前）：

> ▼ このコードがやること:

```
# ※説明用の簡易例
# キー:値 のペアを波カッコで
person = {"name": "admin", "no": 12345}
# → "admin" （"name"というキーで引く）
print(person["name"])
# → 12345
print(person["no"])
```

- `{ }`　　`[ ]`
- `"name": "admin"`　　`"name"`　　`"admin"`
- `person["name"]`

　　`main_views.py`

```
data = {
  # キー "name" → 値 "業務工数システム"
  "name": "業務工数システム",
  # キー "short_name" → 値 "業務工数"
  "short_name": "業務工数",
  # キー "start_url" → 値 "/"
  "start_url": "/",
}
```

> **リストと辞書の使い　け:**　　`numbers[0]`
>
> 　　`person["name"]`
>
> 　　`07`

　　`.get()`

```
# ※説明用の簡易例
# → "admin"（キーがあれば値）
person.get("name")
# → None（キーがなくてもエラーにならず None を返す）
person.get("age")
```

```
print(df["age"]
```

```
```





4.2　較の記号（　較演算子）

記号	意	
`==`		`x == 5`
`!=`		`x != 5`
`>` `>=`		`x >= 80`
`<` `<=`		`x < 0`

`asynchronous_views.py`

```
# 終了日が今日以降なら
if end_date_obj >= today_date_obj:
    ...
# 開始日が終了日より後なら
if start_date_obj > end_date_obj:
    ...
```

4.3 条件をつなぐ（and / or / not）

記号	意	たとえ
`and`		かつ
`or`		または
`not`		でない

```
# ※説明用の簡易例
# x が 0 より大きく、かつ 100 より小さい
if x > 0 and x < 100:
    print("範囲内")
```

`asynchronous_views.py`

```
# start_day が無い、または end_day が無いなら
if not start_day or not end_day:
    return JsonResponse(...)
```

- `not start_day`
- `or`

> 語：truthy（トゥルーシー）/ falsy（フォルシー）　`if 変数:` の
> #　`""`　`0`　`None`　`[]`　falsy（偽とみなす）　truthy（真とみなす）　`if not start_day:`
> `main_utils.py`
> `if not value:`

4.4　在チェック（in / is）

```

```

>

---

5. 繰り　し for

5.1 for の基　— リストを順　に処

> **繰り　し（ループ）とは？**

> **▼ このコードがやること:**

```
# ※説明用の簡易例
numbers = [10, 20, 30]
# numbers から1つずつ取り出して n に入れる
for n in numbers:
    # → 10, 20, 30 が順に表示される
    print(n)
```

- `for 変数 in リスト:`　　`変数`

- `n`

5.2　　のコードで見る for

`main_views.py`

```
# 班員の番号を1つずつ m に
for m in valid_member_numbers:
  follow_message = ''
  # その番号の人がいたら
  if member.objects.filter(employee_no=m).exists():
    member_name = member.objects.get(employee_no=m).name
    # 過去7日を1日ずつ d に（ループの中のループ）
    for d in day_list:
      ...
```

-　　`for m in valid_member_numbers:`

- `for d in day_list:`
- **ループの　にループ（　れ子）**

5.3 range（　　を作る）

> **▼ このコードがやること:**

`main_views.py`

```
day_list = [today - datetime.timedelta(days=d) for d in range(1, 8)]
```

- `range(1, 8)`　`1, 2, 3, 4, 5, 6, 7`　　**8は含まない**
-

> ⚠️ **`range(1, 8)` は8を含まない:**　　`range(開始, 終了)`　**終　の手前まで**
> `range(1, 8)`　　`range(1, 8)`

5.4 ループを止める break

```
# 本物のコード（main_views.py TeamMenu より）
for d in day_list:
  if Business_Time_graph.objects.filter(employee_no3=m, work_day2=d).exists():
    ...
    if not kosu_get.judgement:
      follow_message = f'{member_name}氏の工数未入力があります。'
      follow_message_list.append(follow_message)
      # 1件見つけたら、このループを途中で抜ける
      break
```

- `break`　　**で　ち切る**

---

6. リスト　　記

　　リストを1行で作る

6.1 基　の形

> リスト　　記（ないほうひょうき／list comprehension）とは？ `[ 式 for 変数 in リスト ]`

```
# ※説明用の簡易例（普通の書き方）
# 空リスト
squares = []
# 1個ずつ取り出して
for n in [1, 2, 3]:
    # 2乗を追加
    squares.append(n * n)
# squares は [1, 4, 9]
```

```
# ※説明用の簡易例（内包表記）
# [1, 4, 9]
squares = [n * n for n in [1, 2, 3]]
```

　　「右から左」 `[1, 2, 3]`　　`n`　　`n * n`

6.2 条件　きの　　記

`if`

> ▼ このコードがやること:

　　`main_views.py`

```
valid_member_numbers = [
  num for num in member_numbers if num is not None and num != ''
]
```

- `member_numbers`　　`num`
- `if num is not None and num != ''`
- 　　`valid_member_numbers`

　　されている　員　号だけを　き出したリスト

6.3　　リストの　　記（実コード）

```
day_list = [today - datetime.timedelta(days=d) for d in range(1, 8)]
```

- `range(1, 8)`　　`d`
- `today - datetime.timedelta(days=d)`
- 　　`day_list`

> 語：`datetime.timedelta(days=d)`　　`today - timedelta(days=1)`

6.4　の　　記（models.py）

`models.py`

```
def_list = [(x, x) for x in "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwx$"]
```

- 　　1　字ずつ　　`x`

- `(x, x)`
- `[('A','A'), ('B','B'), ...]` 1 字ずつ手で書く手間を省いて

---

7. 関数 def（引数・初期 ・ り ）

7.1 関数とは

> 関数（かんすう／function）とは？ 前を けて

> ▼ このコードがやること:

```
# ※説明用の簡易例
# add という名前の関数を定義。a と b を受け取る
def add(a, b):
    # 計算
    result = a + b
    # 結果を返す
    return result

# 関数を呼び出す。answer は 8
answer = add(3, 5)
```

- `def 関数名(引数):` `def` `:`
- `a, b` 引数（ひきすう）
- `return 値` り （もどりち） `return` `add(3,5)`
- `add(3, 5)` 呼び出し `a=3, b=5`

> 語：引数（ひきすう） 語： り （もどりち／ り

```
)
```

7.2 引数の初期　（デフォルト引数）

　　あらかじめ　を設

```
# ※説明用の簡易例
# greeting に初期値
def greet(name, greeting="こんにちは"):
    print(greeting + name + "さん")

# → こんにちは田中さん（greeting 省略 → 初期値が使われる）
greet("田中")
# → おはよう田中さん（指定すればそちら）
greet("田中", "おはよう")
```

- `greeting="こんにちは"`　　初期　（デフォルト　）
-

　　`main_views.py`

```
# mode が無ければ 'day'
mode = request.query_params.get('mode', 'day')
```

- `.get('mode', 'day')`　　`'day'`

7.3 引数なし・　り　なしの関数

　　`main_views.py`

```
# request だけ受け取る
def get_logs(request):
  with open('web_console.log', 'r') as log_file:
    logs = log_file.readlines()
  # JSONレスポンスを返す
  return JsonResponse({'logs': logs})
```

`main_utils.py`

```
# 引数なし（カッコの中が空）
def get_all_model_names_in_myapp():
  model_names = []
  ...
  return model_names
```

7.4　数の　を　す（タプルで　す）

数の　をー　に　せます

`main_utils.py`

```
def validate_employee_no_logic(value, member_model):
  if not value:
    # 2つの値（True と None）を返す
    return True, None
  ...
  # 2つの値を返す
  return False, 'は自然数で入力して下さい'
```

- `return True, None`　　**2つを　時に　す**
- 　　`成功, メッセージ = validate_...()`

---

8. 三項演算子・f　字

8.1 三項演算子（条件を1行で書く）

> **三項演算子（さんこうえんざんし）とは？** `A if 条件 else B`

```
# ※説明用の簡易例（普通の if）
if score >= 60:
    result = "合格"
else:
    result = "不合格"
```

```
# ※説明用の簡易例（三項演算子）
result = "合格" if score >= 60 else "不合格"
```

`"合格"` `score >= 60` `"不合格"`

`main_views.py`

```
team_get = team_filter.first() if team_filter.exists() else None
```

- `main_views.py`

```
judgement = True if search_judgement == 'OK' else False
```

- `signals.py`

```
session_data = request.session.get('login_No') if request else None
```

-

8.2 f　字　（　字の　に　を　め込む）

> **f　字　（エフもじれつ／f-string）とは？**　　`f`　　`{ }`

```
# ※説明用の簡易例
name = "田中"
# → "田中さん、こんにちは"
message = f"{name}さん、こんにちは"
```

- 　　`f`
- `{name}`
- `f`　　`{name}`

　　`main_views.py`

```
follow_message = f'{member_name}氏の工数未入力があります。'
```

- `{member_name}`

　　`models.py`　　`__str__`

```
return f'{self.created_at} on {self.status} (TaskID: {self.task_id})'
```

-

> **`{ }`　の　には式も書ける:** `f"合計は{a + b}円"`
>
> `asynchronous_views.py`　　`f'ファイル書き込みエラー: {str(e)}'`

```
str(e)
```

---

9. クラスとインスタンス（self / init / str / 継承）

クラスだらけ

9.1 クラスとは「設計図」、インスタンスとは「製　」

> クラス（class：設計図）とは？
> 　設計図　インスタンス（instance：実　）とは？　実際のモノ

> たとえ　：たい焼き クラス　金型
> 　インスタンス　1個1個のたい焼き

> ▼ このコードがやること:

```python
# ※説明用の簡易例
# 「人」の設計図（クラス）
class Person:
    # 作るときに呼ばれる初期化メソッド
    def __init__(self, name, age):
        # この人の名前を記録
        self.name = name
        # この人の年齢を記録
        self.age = age

    # この人ができること（メソッド）
    def greet(self):
        print(f"私は{self.name}です")

# 設計図から「田中さん」を作る（インスタンス化）
tanaka = Person("田中", 30)
# → 私は田中です
tanaka.greet()
```

> **語：メソッド（method）**
>
> `greet` `self`

9.2 `__init__`（初期化メソッド）

> **`__init__`（イニット）とは？** **自　で呼ばれる**
>
> `__`

`Person("田中", 30)` `__init__(self, "田中", 30)`

`main_utils.py` `__init__`

```
class CustomPagination(PageNumberPagination):
  page_size = 20

  # 作られるとき自動実行
  def __init__(self):
    last_record = administrator_data.objects.order_by("id").last()
    if last_record is not None:
      # 1ページの表示件数を設定から取得
      self.page_size = last_record.menu_row
```

- 管　設　から「1ページに何件　示する
か」を　み込んで

9.3 `self`（自　自身）

> `self`（セルフ：自　自身）とは？
> この

- 1引数は　ず `self`
- `self.name`　`tanaka.name`　`suzuki.name`

> なぜ self が　？
> `self`
> 呼び出すときは self を書きません　`tanaka.greet()`　`tanaka.greet(tanaka)`

`main_views.py`

```
class PrintToLogger:
  # 第1引数は self（自分自身）
  def write(self, message):
    if message.strip():
      views_logger.info(message)

  # こちらも第1引数は self
  def flush(self):
    pass
```

9.4 __str__（　字　　現）

> __str__（ストア）とは？　　字として　示したいとき
>
> print(インスタンス)

　　models.py

```
class member(models.Model):
  ...
  def __str__(self):
    # この人を表示するときは「氏名」を出す
    return self.name
```

-
  氏

　　models.py　　__str__

```
def __str__(self):
  return str(self.id) + '__' + str(self.work_day2) + ':' + str(self.employee_no3)
```

- str(...)　　+

- 　42__2026-06-01:12345

> 語：`+` による　字　　結　　`+`
> `"あ" + "い"`　`"あい"`　　数　と　字はそのまま `+` できない
> `str(self.id)`

9.5 継承（けいしょう）— 設計図を引き継ぐ

> 継承（inheritance）とは？　　引き継いで

> たとえ　：

`class 子クラス(親クラス):`

`models.py`　　`models.Model`

```
# models.Model を継承
class member(models.Model):
  ...
```

- `(models.Model)`
- 　　データベースに　　・検索・削除する機　をタダで
  `.objects.get()`
  `07`

`main_utils.py`

```
# 既製のページ送り機能を継承
class CustomPagination(PageNumberPagination):
  page_size = 20
  def __init__(self):
    # 一部だけ自分用にカスタマイズ
    ...
```

継承が出てくる `super()`：

`models.py`

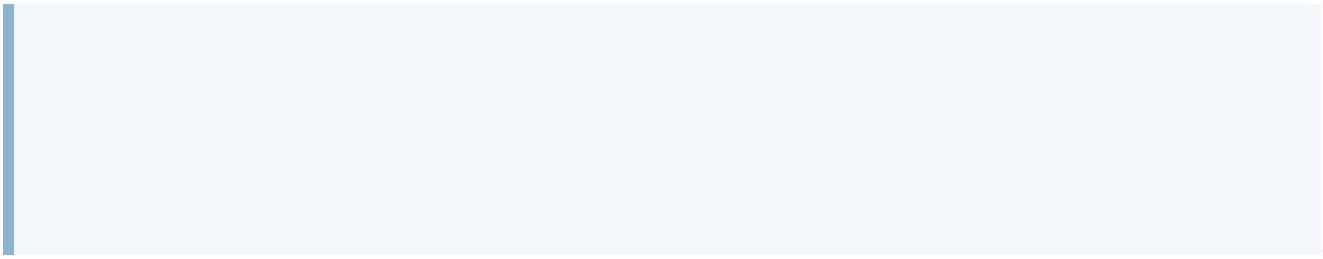
```
def save(self, *args, **kwargs):
  super().save(*args, **kwargs)     # まず親（Model）の保存処理を実行
  ...                               # その後、自分の追加処理（古いレコード削除）
```

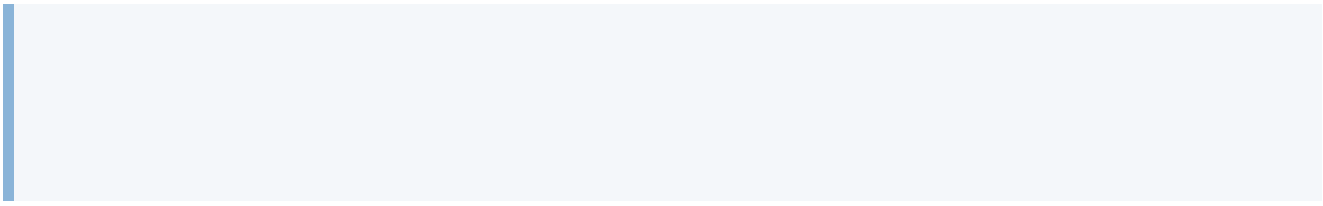
`*args, **kwargs`

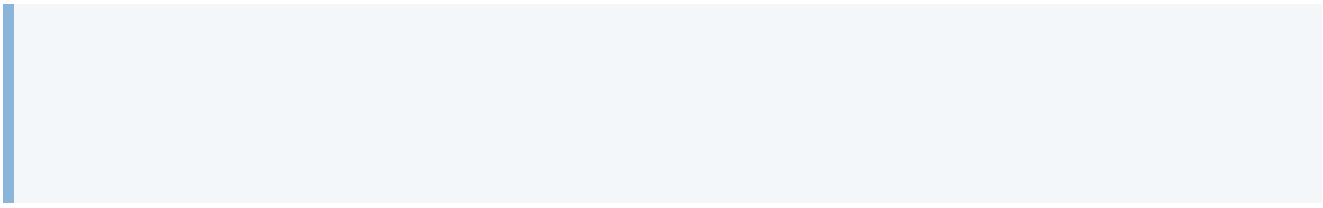
,

10.　外処　try-except

T01 1　OO☒







```
# ※説明用の簡易例
try:
    # "abc" を数値に変換しようとする（失敗する）
    value = int("abc")
# ValueError（値の変換エラー）が起きたら
except ValueError:
    # こう対処する（止まらない）
    print("数値ではありません")
```

- `try:`
- `except エラー名:`
- `int("abc")`

10.3 のコードで見る try-except

`main_utils.py`

```
try:
  # 数値に変換を試みる
  value_int = int(value)
# 変換できなければ（例: 'abc'）
except ValueError:
  return False, 'は自然数で入力して下さい'
```

- `int(value)`
  `ValueError`
- `except`
  ミスでサーバーが止まらない

`main_views.py`

```
try:
  # 送られたデータをPythonの辞書に変換
  data = json.loads(request.body)
  input_number = data.get('employee_no')
# データが壊れた形式なら
except json.JSONDecodeError:
  return JsonResponse({'status': 'error', 'message': 'JSON形式が正しくありません。'},
...)
```

10.4 Djangoでよく見る「データが　い」　外

`.get()`

`main_views.py`

```
try:
  # 該当する人を1人取得
  member_data = member.objects.get(employee_no=login_no)
# その人がいなければ
except member.DoesNotExist:
  return Response({'status': 'error', 'message': 'ユーザー情報が見つかりません。'},
...)
```

- `member.objects.get(...)`　　1件取
- 　　`member.DoesNotExist`
-

10.5 何でも　まえる except Exception

`asynchronous_views.py`

```
# どんなエラーでも捕まえる
except Exception as e:
  ...
  # エラー内容を文字にして記録
  task.result = str(e)
```

- `Exception`　すべての　外の親　`except Exception:`
- `as e`　　`e`　　`str(e)`
-

> ⚠️ `except Exception` の　は　意:
>
> `str(e)`

---

11. with　（ファイルを安全に扱う）

11.1 with とは

> `with`　とは？　　自　で後
> けしてくれる

ず　じる
`with`　　自　で　じて

11.2　のコードで見る with

`main_views.py`

```
# ログファイルを開く
with open('web_console.log', 'r') as log_file:
    # 全行をリストで読み込む
    logs = log_file.readlines()
# ← ここでブロックを抜けると、log_file は自動で閉じられる
```

- `as log_file` `log_file`
- `with ...:` 字下げされた 自 でクローズ
- `.readlines()`

> なぜ with を使う？ `with` `log_file.close()`
> `with` エラーが起きても確実に じて

11.3 の with（一時ファイルの作成）

`asynchronous_views.py`

```
with tempfile.NamedTemporaryFile(suffix=".xlsx", delete=False) as temp_file:
  # アップロードされたファイルを小分けに
  for chunk in kosu_file.chunks():
    # 一時ファイルに書き込む
    temp_file.write(chunk)
  # 一時ファイルのパスを覚えておく
  temp_file_path = temp_file.name
```

- `tempfile.NamedTemporaryFile(...)`
- `for chunk in kosu_file.chunks():` 小さなかたまり（chunk）ずつ
- `with`

> `mode='r'` 以外のモード: `'r'` `'w'` `'a'`
> `main_views.py` `open(LOG_FILE_PATH, 'r', encoding='utf-8')` 字コード（encoding）



12.2　準ライブラリと外　ライブラリ

- **　準ライブラリ**　　`os` `json` `datetime` `inspect`

- **外　ライブラリ**　　`django` `rest_framework` `environ`
  　　`pip install`　　`01`

　　`main_utils.py`

```
# 標準：オブジェクトを調べる道具
import inspect
# 外部：DRFのページ送り
from rest_framework.pagination import PageNumberPagination
# 外部：Djangoのモデル機能
from django.db import models
```

12.3 相　import（自　のアプリ　のファイルを　む）

> **相　import（そうたいインポート）とは？ ドット `.`**

書き	意
`.module`	じ階層
`..module`	1つ上の階層

　　`main_views.py`

```
from ..models import member, Business_Time_graph, kosu_division, ...
from .serializers import MemberSerializer, ...
from ..utils.main_utils import CustomPagination, get_all_model_names_in_myapp
```

　　`main_views.py`　`kosu/views/`

- `from ..models import member`　`..`　　`kosu/`　`models.py`　`member`
  `views/`　　`models.py`　　`..`

- `from .serializers import ...` `.` `kosu/views/` `serializers.py` `.`
- `from ..utils.main_utils import ...` `..` `kosu/` `utils` `main_utils.py`

> **なぜ相 import？** `from kosu.views.serializers import ...` `.` `..`

`asynchronous_views.py`

```
from ..tasks import generate_kosu_backup, delete_kosu_data, load_kosu_file, \
                    generate_member_backup, load_member_file, ...
```

> **行 の `\`（バックスラッシュ）：**
> `\`

---

13. デコレータ @

13.1 デコレータとは

> **デコレータ（decorator）とは？** **真上に `@名前` と書いて**

ラッピング `@`

13.2 のコードで見るデコレータ

`asynchronous_views.py`

```
# ← デコレータ
@api_view(['POST'])
# ← デコレータ
@parser_classes([MultiPartParser, JSONParser, FormParser])
# この関数に上の性質が付く
def backup(request):
  ...
```

- `@api_view(['POST'])`

- `@parser_classes([...])`

- **関数の真上に書く**

13.3 signals.py の @receiver（最重　デコレータ）

`signals.py`

```
# ← デコレータ
@receiver(post_save, sender=member)
def log_create_update_member_history(sender, instance, created, **kwargs):
  ...
```

- `@receiver(post_save, sender=member)` **member が　（post_save）された ら自　で呼ばれる**
- `member.save()` **自　で呼ばなくても**

- `post_save` `pre_save` `post_delete`

> **なぜデコレータが　？**
>
> `@receiver` **どこで　され ても自　で**

13.4　にもあるデコレータ

> **`@property` など:**　　`@property`
> `@staticmethod`
> `@api_view`　　`@receiver`

---

14. 実コード全解説　：main_views.py の Login・PrintToLogger・get_logs

　　1行ずつ

`kosu/views/main_views.py`

14.1 import　　（1〜17行　）

> **▼ このブロックがやること:**

```python
# 日付・時刻を扱う標準ライブラリ
import datetime
# ファイルパスなどOS機能の標準ライブラリ
import os
# システム関連（標準出力の差し替えに使う）
import sys
# ログ記録の標準ライブラリ
import logging
# .envファイル（環境変数）を読む外部ライブラリ
import environ
# JSONの読み書きの標準ライブラリ
import json
# パス操作の道具 Path を取り出す
from pathlib import Path
# HTMLを描画する関数
from django.shortcuts import render
# JSON形式で応答を返すクラス
from django.http import JsonResponse
# Djangoの基本ビュークラス
from django.views import View
# 設定（settings.py）を読む
from django.conf import settings
# DRFのAPI用ビュークラス
from rest_framework.views import APIView
# DRFの応答クラス
from rest_framework.response import Response
# HTTPステータス番号の定数集
from rest_framework import status
# 自作モデル
from ..models import member, Business_Time_graph, kosu_division, team_member,
administrator_data, AsyncTask, History
# 自作シリアライザ
from .serializers import MemberSerializer, AdministratorSerializer, KosuSerializer,
DefSerializer, TaskSerializer, HistorySerializer
# 自作の便利部品
from ..utils.main_utils import CustomPagination, get_all_model_names_in_myapp
```

-

- `..models` `.serializers` `..utils.main_utils` **相 import**

> **語：ロガー（logger）** `logging`
>
> **語：シリアライザ（serializer）**

> 換する係　07

14.2 ロガーの取　（22〜23行　）

```
# 専用ロガー取得('views_logger'という名前のカスタムロガーを使用しログメッセージ記録)
views_logger = logging.getLogger('views_logger')
```

- `logging.getLogger('views_logger')`　`'views_logger'`
  `views_logger`
- 　`views_logger.info("メッセージ")`

14.3 PrintToLogger クラス（27〜38行　）

> ▼ このブロックがやること:　`print()`
> ログに記　`print`

```
# クラス定義（§9）
class PrintToLogger:
  # write メソッド（self は自分自身）
  def write(self, message):
    # 受け取った標準出力のメッセージ処理(空白および改行のみのメッセージは無視)
    # 空白・改行だけのメッセージは無視
    if message.strip():
      # 中身があればログに記録
      views_logger.info(message)

  # flush メソッド（必要だが処理は不要）
  def flush(self):
    # 処理不要のため空処理
    # 「何もしない」を表すキーワード
    pass
```

- `class PrintToLogger:`

- `def write(self, message):` `write` `print()` `write` `self` `message`
- `if message.strip():` `.strip()`
- `views_logger.info(message)`
- `def flush(self):` `flush`
- `pass` 何もしない

> 語：`pass`

14.4 準出 の差し え（37〜38行 ）

```
# 標準出力をロガーへリダイレクト(sys.stdoutをPrintToLoggerのインスタンスに設定)
sys.stdout = PrintToLogger()
```

- `PrintToLogger()` インスタンスを作る
- `sys.stdout = ...` `sys.stdout` すり える
- `print("xxx")` ログに残る

14.5 get_logs 関数（41〜47行 ）

> ▼ このブロックがやること:

```
# request を受け取る関数（§7）
def get_logs(request):
  # 'web_console.log' ファイルを読み込み専用モードで開く
  # ファイルを開く（with §11）
  with open('web_console.log', 'r') as log_file:
    # ファイルの内容を行ごとにリストとして格納
    # 全行をリストで読み込む
    logs = log_file.readlines()
  # JSONレスポンスとして、ログを返す
  # 辞書をJSONにして返す（§3.2 §7.3）
  return JsonResponse({'logs': logs})
```

- `def get_logs(request):` `request`
- `with open('web_console.log', 'r') as log_file:` `with`
- `logs = log_file.readlines()` `logs`
- `return JsonResponse({'logs': logs})` `{'logs': logs}` 辞書 `'logs'`

> **▼ こう れば成功:**
>
> ```
> {"logs": ["1行目のログ\n", "2行目のログ\n", "..."]}
> ```

14.6 Login クラス（79〜102行 ）— 章の主

> **▼ このブロックがやること:**

```
# ログイン
# APIView を継承（§9.5）
class Login(APIView):
  # POST処理
  # POSTで呼ばれるメソッド
  def post(self, request):
    # まず試す（§10）
    try:
      # 送られてきたデータ確認
      # 受信データをPython辞書に変換
      data = json.loads(request.body)
      # その中の従業員番号を取り出す
      input_number = data.get('employee_no')
    # データが壊れていたら
    except json.JSONDecodeError:
      return JsonResponse({'status': 'error', 'message': 'JSON形式が正しくありません。'}, status=status.HTTP_400_BAD_REQUEST)

    # POSTされた従番が人員データにある場合、セッションに保存&最新工数区分定義取得
    # その番号の人がいるか
    if member.objects.filter(employee_no=input_number).exists():
      # セッションに番号を保存
      request.session['login_No'] = input_number
      # 最新の工数区分定義を取得
      def_Ver = kosu_division.objects.order_by("id").last()

      # 取得した工数区分定義をセッションに保存
      # 定義が1つも無ければ
      if not def_Ver:
        return JsonResponse({'status': 'error', 'message': '利用可能な工数区分がありません。ERROR052'}, status=status.HTTP_400_BAD_REQUEST)
      # 定義があれば
      else:
        # 定義名をセッションに保存
        request.session['input_def'] = def_Ver.kosu_name
        # 成功を返す
        return JsonResponse({'status': 'success'})

    # その番号の人がいなければ
    else:
      return JsonResponse({'status': 'error', 'message': '入力された従業員番号は登録がありません。ERROR048'}, status=status.HTTP_400_BAD_REQUEST)
```

ほぼ全

- `class Login(APIView):` `Login` `APIView` **継承**

- `def post(self, request):` `post` POSTで
  `self` `request`
- `try:`
- `data = json.loads(request.body)` `request.body`
  `json.loads` 辞書 `loads`

- `input_number = data.get('employee_no')` `'employee_no'`
  `.get()`
- `except json.JSONDecodeError:`

- `return JsonResponse({...}, status=...)`
  `status=status.HTTP_400_BAD_REQUEST`
  `07`
- `if member.objects.filter(employee_no=input_number).exists():` `member`
  `.exists()`

- `request.session['login_No'] = input_number`
  `request.session`
- `def_Ver = kosu_division.objects.order_by("id").last()` id順に べて最後の1件
- `if not def_Ver:` `def_Ver`

- `else:` `def_Ver.kosu_name` `{'status': 'success'}`
- `else:`

> 語：セッション（session）
> `request.session['login_No']`
> `07`

> 語：`.filter()` と `.get()` と `.exists()`（Djangoのデータ操作）
>
> - `.filter(条件)` 数

- `.get(条件)` 1件だけ
- `.exists()`
- `.order_by("id").last()` `.first()`
  `07`

14.7 TeamMenu の　員チェック（340〜378行　）— for・　　記・f　字　の合

▼ このブロックがやること:

```
# 班員フォローメッセージ作成
# 空のリスト（§3.1）
follow_message_list = []
# 班員データあるか確認
team_filter = team_member.objects.filter(employee_no5=login_no)
# 三項演算子（§8.1）
team_get = team_filter.first() if team_filter.exists() else None

# 班員フォロー有効の場合、班員リスト作成
# and で2条件（§4.3）
if team_filter.exists() and team_get.follow:
  # 15人分の番号をリストに
  member_numbers = [
    team_get.member1, team_get.member2, team_get.member3,
    team_get.member4, team_get.member5, team_get.member6,
    team_get.member7, team_get.member8, team_get.member9,
    team_get.member10, team_get.member11, team_get.member12,
    team_get.member13, team_get.member14, team_get.member15
  ]
  # 内包表記（§6.2）
  valid_member_numbers = [
    num for num in member_numbers if num is not None and num != ''
  ]

  # 過去7日分の日付リスト作成
  # 今日の日付
  today = datetime.date.today()
  # 内包表記（§6.3）
  day_list = [today - datetime.timedelta(days=d) for d in range(1, 8)]

  # 班員ごとに工数未入力確認
  # 班員を1人ずつ（§5）
  for m in valid_member_numbers:
    follow_message = ''
    # その人がいるか
    if member.objects.filter(employee_no=m).exists():
      # 名前を取得
      member_name = member.objects.get(employee_no=m).name
      # 7日を1日ずつ（入れ子ループ）
      for d in day_list:
        if Business_Time_graph.objects.filter(employee_no3=m, work_day2=d).exists():
          kosu_get = Business_Time_graph.objects.get(employee_no3=m, work_day2=d)
          # 判定がOKでなければ
          if not kosu_get.judgement:
            # f文字列（§8.2）
            follow_message = f'{member_name}氏の工数未入力があります。'
            # リストに追加（§3.1）
            follow_message_list.append(follow_message)
```

```
            # このループを抜ける（§5.4）
            break
        # その日のデータ自体が無ければ
        else:
            follow_message = f'{member_name}氏の工数未入力があります。'
            follow_message_list.append(follow_message)
            break
```

- `follow_message_list = []` `.append()`
- `team_get = ... if ... else None`
- `member_numbers = [...]`
- `valid_member_numbers = [num for num in ... if ...]`

- `day_list = [... for d in range(1, 8)]`
-
- `f'{member_name}氏の...'`
- `break`

1つのコードブロックの　に、リスト・辞書アクセス・三項演算子・　　記・　重ループ・f　字　・break が全　詰まっています。

---

15. 実コード全解説　：main_utils.py の検証関数

`kosu/utils/main_utils.py`

15.1 import（1〜6行　）

```
# オブジェクトの中身を調べる標準ライブラリ
import inspect
# DRFのページ送り機能
from rest_framework.pagination import PageNumberPagination
# DRFの応答クラス
from rest_framework.response import Response
# Djangoのモデル基本機能
from django.db import models
# 自作モデルを myapp_models という別名で
from kosu import models as myapp_models
# 設定モデルを相対importで
from ..models import administrator_data
```

- `from kosu import models as myapp_models`　`as`　　（あだ　）を　ける　`kosu.models`　`myapp_models`

15.2 CustomPagination クラス（11〜25行　）

> **▼ このブロックがやること:**

```python
# ページネーションクラス
# 既製のページ送りを継承（§9.5）
class CustomPagination(PageNumberPagination):
  # デフォルトの設定値              # クラス変数（初期値20件）
  page_size = 20

  # 作られるとき自動実行（§9.2）
  def __init__(self):
    # administrator_data から動的にページサイズを設定
    # 最新の設定を取得
    last_record = administrator_data.objects.order_by("id").last()
    # 設定があれば（§4.4）
    if last_record is not None:
      # 1ページの件数を設定値に上書き
      self.page_size = last_record.menu_row

  # ページ送り済みの応答を作るメソッド
  def get_paginated_response(self, data):
    # 辞書を応答にして返す（§3.2）
    return Response({
      # 合計件数
      'count': self.page.paginator.count,
      # ページサイズ
      'page_size': self.page_size,
      # 現在のページのデータ
      'results': data,
    })
```

- `class CustomPagination(PageNumberPagination):` `PageNumberPagination` **継承**
- `page_size = 20` **クラス 数**
- `def __init__(self):`
- `last_record = administrator_data.objects.order_by("id").last()`
- `if last_record is not None:` `is not None`
- `self.page_size = last_record.menu_row` `self.page_size` `menu_row` `self.`

- `def get_paginated_response(self, data):`
- `return Response({...})`

> なぜ設　から件数を　む？
>
> `20`
>
> 画　から　えられる

15.3 validate_employee_no_logic 関数（29〜50行　）

> ▼ このブロックがやること:

- `if value_int <= 0:`
- `try: member_model.objects.get(employee_no=value_int)`
  `.get()`
- `return True, value_int`
- `except member_model.DoesNotExist: return False, '...'`

> **この関数の設計の　しさ:**
> `return 成否, 理由`　　じ形　　`ok, msg = validate_employee_no_logic(...)`　　`if not ok:`

15.4 get_all_model_names_in_myapp 関数（56〜68行　）

> **▼ このブロックがやること:**

```
# 引数なしの関数（§7.3）
def get_all_model_names_in_myapp():
  # 空のリスト（§3.1）
  model_names = []

  # myapp_modelsモジュールのメンバーを全て取得
  # models.py の中身を1つずつ
  for name, obj in inspect.getmembers(myapp_models):
    # それがクラスで（§4.3 and）
    if (inspect.isclass(obj) and
      # models.Model を継承していて
      issubclass(obj, models.Model) and
      # 抽象クラスでなく
      not obj._meta.abstract and
      # プロキシでなく
      not obj._meta.proxy and
      # この自作モジュール由来である
      obj.__module__ == myapp_models.__name__):
      # 条件を満たせば名前をリストに追加
      model_names.append(name)

  # 集めた名前リストを返す
  return model_names
```

- `def get_all_model_names_in_myapp():`
- `model_names = []`
- `for name, obj in inspect.getmembers(myapp_models):` `inspect.getmembers` `name` `obj`

for で2つの　数に　時に受け取る

- `if (... and ... and ...):` `and`
  - `inspect.isclass(obj)`
  - `issubclass(obj, models.Model)` `models.Model`
  - `not obj._meta.abstract` `not`
  - `not obj._meta.proxy`
  - `obj.__module__ == myapp_models.__name__`

- `model_names.append(name)`
- `return model_names`

> **なぜこんな自　化を？**
>
> `main_views.py`　　`get_all_model_names_in_myapp()`
>
> 　**モデルを　やしても選　肢が自　で　える**　`inspect`

> **語：　象クラス（abstract）/ プロキシ（proxy）**
>
> 　　`not`

---

16. 実コード全解説　：asynchronous_views.py の validate_dates

`kosu/views/asynchronous_views.py`　　`validate_dates`
`07`

16.1 validate_dates 関数（200〜215行　）

> **▼ このブロックがやること:**

```
# 日付バリデーション関数
# 2つの日付を受け取る
def validate_dates(start_day, end_day):
  # 今日を 'YYYY-MM-DD' の文字に
  today_str = datetime.date.today().strftime('%Y-%m-%d')
  # どちらか空なら（§4.3 or）
  if not start_day or not end_day:
    return JsonResponse({'status': 'error', 'message': '日付を指定してください。'},
status=status.HTTP_400_BAD_REQUEST)

  # 日付変換を試す（§10）
  try:
    # 文字→日付オブジェクト
    end_date_obj = datetime.date.fromisoformat(end_day)
    today_date_obj = datetime.date.fromisoformat(today_str)
    start_date_obj = datetime.date.fromisoformat(start_day)
  # 形式が不正なら
  except ValueError:
    return JsonResponse({'status': 'error', 'message': '日付の形式が不正です。'},
status=status.HTTP_400_BAD_REQUEST)
  # 終了日が今日以降なら（§4.2）
  if end_date_obj >= today_date_obj:
    return JsonResponse({'status': 'error', 'message': '昨日の日付までしか指定できませ
ん。'}, status=status.HTTP_400_BAD_REQUEST)
  # 開始日が終了日より後なら
  if start_date_obj > end_date_obj:
    return JsonResponse({'status': 'error', 'message': '開始日が終了日を超えていま
す。'}, status=status.HTTP_400_BAD_REQUEST)
  # 全部OKなら None（問題なし）を返す
  return None
```

- `def validate_dates(start_day, end_day):`
- `today_str = datetime.date.today().strftime('%Y-%m-%d')` `datetime.date.today()` `.strftime('%Y-%m-%d')` 字 `%Y` `%m` `%d`
- `if not start_day or not end_day:`
- `return JsonResponse({...}, status=...)`
- `try:`

- `datetime.date.fromisoformat(end_day)`
  オブジェクト　　`fromisoformat`
- `except ValueError:`
- `if end_date_obj >= today_date_obj:`　　`>=`　　小　較
- `if start_date_obj > end_date_obj:`
- `return None`　　`None`

> この関数の `None` の意　：　　があればエラー応　を、　　なければ None を
> す
>
> ```
> error_response = validate_dates(start_day, end_day)
> if error_response:                  # None でなければ（＝エラーがあれば）
>   return error_response             # そのエラーをそのまま返す
> ```
>
> `if error_response:`

> 語：　　オブジェクト `datetime.date`
>
> `fromisoformat`

16.2 デコレータ　きの backup 関数の　（21〜27行　）

```
# API化デコレータ（§13.2）
@api_view(['POST'])
# データ形式指定デコレータ
@parser_classes([MultiPartParser, JSONParser, FormParser])
def backup(request):
  # タスクID生成
  # 一意なIDを文字列で生成
  task_id = str(uuid.uuid4())
  # 処理中レコードを作成
  AsyncTask.objects.create(task_id=task_id, status='pending')

  # 開始日を取得（§3.2 .get）
  start_day = request.data.get('start_day')
  # 終了日を取得
  end_day = request.data.get('end_day')
```

- `@api_view(['POST'])`
- `str(uuid.uuid4())` `uuid.uuid4()`
  `str(...)`
- `AsyncTask.objects.create(...)`

- `request.data.get('start_day')`

> `url_name`
> `if/elif`
> `threading` `07`

---

17. 実コード全解説　：models.py の member クラスの形

`kosu/models.py` `member` **データベースのテーブルの設計図**





```
# 整数の列
employee_no = models.IntegerField('従業員番号')
# 短い文字の列（最大100字）
name = models.CharField('氏名', max_length=100)
# 選択肢付き文字
shop = models.CharField('ショップ', choices = shop_list, max_length=15)
# 真偽値の列（True/False）
authority = models.BooleanField('権限')
# 真偽値の列
administrator = models.BooleanField('管理者')
# 休憩時間（文字9字）
break_time1 = models.CharField('1直昼休憩時間', max_length=9)
break_time1_over1 = models.CharField('1直残業休憩時間1', max_length=9)
...
pop_up1 = models.CharField('ポップアップ1', max_length=255, null=True, blank=True)
...
break_check = models.BooleanField('休憩エラー有効チェック', null=True)
```

- `フィールド名 = models.〇〇Field('表示名', オプション)`
- `models.IntegerField(...)` 整数
- `models.CharField('...', max_length=100)` い 字 `max_length`
- `models.BooleanField('...')` 真偽
- `choices=shop_list` 選 肢に限
- `null=True` なし（None）を許可
- `blank=True` 空 を許可
- `'従業員番号'` 語の 前（ラベル）

> 語：フィールド（field）
>
> 語：
>
> モデル（model）
>
> `07`

> なぜ `null=True` と `blank=True` の　？ `null`　データベース　　`blank` フォーム

17.3 `__str__` メソッド（64〜65行　）

```
  # 文字列表現メソッド（§9.4）
  def __str__(self):
    # この人を表示するときは氏名を出す
    return self.name
```

- `def __str__(self):`
- `return self.name`　　`name`　　`self.name`　`self`
-　　氏

17.4　モデルの `__str__` も見ておく

`models.py`　　`__str__`

```
# Business_Time_graph （文字列連結 §9.4）
def __str__(self):
  return str(self.id) + '__' + str(self.work_day2) + ':' + str(self.employee_no3)

# AsyncTask （f文字列 §8.2）
def __str__(self):
  return f'{self.created_at} on {self.status} (TaskID: {self.task_id})'

# History （f文字列 §8.2）
def __str__(self):
  return f'{self.operation} on {self.table_name} (ID: {self.record_id})'
```

> +
>
> `str()` `f'...{値}...'`

17.5 save をオーバーライドする（継承＋super の実　）

`models.py` `super()`

> **▼ このブロックがやること:**

```
class History(models.Model):
  # 上限件数（クラス変数）
  MAX_RECORDS = 500000
  ...
  # 親の save を上書き（オーバーライド）
  def save(self, *args, **kwargs):
    # まず親（Model）の保存を実行（§9.5）
    super().save(*args, **kwargs)
    # 現在の総件数を数える
    current_count = self.__class__.objects.count()

    # レコード数が許容数以上の場合の処理
    # 上限を超えたら
    if current_count > self.MAX_RECORDS:
      # 超過レコード数分のレコード取得し削除
      # 何件オーバーか
      excess_count = current_count - self.MAX_RECORDS
      # 古い順に超過分
      oldest_records = self.__class__.objects.order_by('timestamp')[:excess_count]
      # 1件ずつ（§5）
      for record in oldest_records:
        # 削除
        record.delete()
```

- `def save(self, *args, **kwargs):` `save`
  上書き（オーバーライド） `*args, **kwargs`

- `super().save(*args, **kwargs)` `super()` 親の save

- `current_count = self.__class__.objects.count()` `self.__class__` `.count()`
- `if current_count > self.MAX_RECORDS:`
- `excess_count = current_count - self.MAX_RECORDS`
- `[:excess_count]` スライス

- `for record in oldest_records: record.delete()`

> 語：オーバーライド（override） じ 前で書き して上書き
>
> 語：スライス（slice） リスト[開始:終了] `[:5]` `[2:]`

---

18. 実コード全解説 ：signals.py の差 計算

`kosu/signals.py` `@receiver`

18.1 import と初期化（1〜11行　）

```
# スレッドごとの保管庫
from threading import local
# 保存・削除の合図
from django.db.models.signals import pre_save, post_save, post_delete
# 合図を受け取る登録用デコレータ
from django.dispatch import receiver
# 自作モデル（相対import §12）
from .models import History, member, Business_Time_graph, ...
# 現在のリクエスト取得
from .middleware.clear_session_middleware import get_current_request
from django.db import models

# スレッドローカル変数初期化
# スレッドごとの一時保管庫を作る
_thread_locals = local()
```

- `from django.db.models.signals import pre_save, post_save, post_delete`
  タイミングの合図（シグナル）
- `from django.dispatch import receiver`　`@receiver`
- `_thread_locals = local()`　**スレッドローカル**

> **語：シグナル（signal）**　　**合図**　`post_save`
>
> **語：スレッド（thread）**

18.2 キャッシュの　　・取　（14〜27行　）

> **▼ このブロックがやること:**

```
# 更新前の値をスレッドローカルキャッシュに保存
# instance（保存される対象）を受け取る
def set_instance_cache(instance):
  # そのオブジェクトのクラス（種類）を取得
  model = type(instance)
  # 試す（§10）
  try:
    # 更新前の値取得→スレッドローカルキャッシュに保存
    # DBの現在値を保管
    _thread_locals.instance_cache = model.objects.get(pk=instance.pk)
  # まだDBに無ければ（新規作成）
  except model.DoesNotExist:
    # レコードなしの場合はNone取得
    # Noneを保管
    _thread_locals.instance_cache = None


# スレッドローカルキャッシュから更新前の値取得
def get_instance_cache():
  # 保管した値を返す（無ければNone）
  return getattr(_thread_locals, 'instance_cache', None)
```

- `model = type(instance)` `type(...)` 型（クラス）

- `_thread_locals.instance_cache = model.objects.get(pk=instance.pk)` `pk` データベースの現在
- `except model.DoesNotExist:`

- `getattr(_thread_locals, 'instance_cache', None)` `getattr(対象, '属性名', 初期値)`

> 語： `pk` （ピーケー／primary key：主キー）
> `id` `instance.pk` 語：キャッシュ（cache）

18.3 差　計算 get_changes（32〜106行　）

> **▼ このブロックがやること:**
>
> `{項目名: {old: 前, new: 後}}`

```
# 保存対象と「新規か(created)」を受け取る
def get_changes(instance, created):
  # 変更点を入れる空の辞書（§3.2）
  changes = {}

  # 1. キャッシュを取得
  # §18.2で保管した更新前の値
  old_instance = get_instance_cache()

  # 【修正】キャッシュが存在しても、現在のインスタンスと型やPKが違う場合は、誤った比較を避けるためNoneにする
  if old_instance and (not isinstance(old_instance, type(instance)) or old_instance.pk != instance.pk):
    # 別物なら比較対象を無効化
    old_instance = None

  # モデルの全フィールド処理
  # 全フィールドを1つずつ（§5）
  for field in instance._meta.fields:
    # フィールド名（例 'name'）
    field_name = field.name

    # 値取得
    # 新しい値を取得（getattrで動的に）
    new_value = getattr(instance, field_name)

    # 新規作成時、全て変更として処理
    # 新規作成なら
    if created:
      # 古い値は無い
      old_value = None
      # 全項目を「変更あり」とする
      is_changed = True
    # 更新時、差分取得
    # 更新なら
    else:
      # 更新前の値があれば
      if old_instance:
        # 古い値を取得
        old_value = getattr(old_instance, field_name)
        # 古い≠新しい なら変更あり（§4.2）
        is_changed = (old_value != new_value)
      else:
        # 【修正】pre_saveでのキャッシュ漏れやスレッドの混同対策としてDBから直接取得を試みる
        try:
          # DBから直接取得
          old_db_instance = type(instance).objects.get(pk=instance.pk)
```

```
            old_value = getattr(old_db_instance, field_name)
            is_changed = (old_value != new_value)
            # 以降のためにキャッシュ更新
            old_instance = old_db_instance
        except type(instance).DoesNotExist:
            # 取れなければこの項目は飛ばす（§18.4）
            continue
```

- `changes = {}` 空の辞書
- `old_instance = get_instance_cache()`
- `if old_instance and (not isinstance(...) or old_instance.pk != instance.pk):`
  `isinstance(A, 型)`
  `old_instance = None`
- `for field in instance._meta.fields:`
  `_meta.fields`
- `new_value = getattr(instance, field_name)` `getattr(対象, '名前')` 字 で 指 して `instance.name`

- `if created:`
- `else:` `!=`
  `is_changed = True`
- `continue`

```
    # 変更or新規作成
    if is_changed:

      # JSON化できないリレーションフィールド処理
      # 他テーブルへの参照なら
      if field.is_relation and field.many_to_one:
        # 関連ID（三項演算子 §8.1）
        old_json_safe_value = getattr(old_instance, field.attname) if old_instance else
None
        new_json_safe_value = getattr(instance, field.attname)

      # その他オブジェクト処理
      # 基本型でなければ
      elif not isinstance(new_value, (str, int, float, bool, type(None))):
        # モデルそのものなら
        if isinstance(new_value, models.Model):
          # 辞書化
          old_json_safe_value = {'id': old_value.pk, 'str': str(old_value)} if
old_value else None
          new_json_safe_value = {'id': new_value.pk, 'str': str(new_value)}
        # 日付なら
        elif isinstance(new_value, (models.DateField, models.DateTimeField)):
          # 文字列化
          old_json_safe_value = old_value.isoformat() if old_value else None
          new_json_safe_value = new_value.isoformat()
        # その他は文字列化
        else:
          old_json_safe_value = str(old_value) if old_value else None
          new_json_safe_value = str(new_value)

      # JSON化可能な基本データ型の処理
      # 数値・文字・真偽値など
      else:
        old_json_safe_value = old_value
        new_json_safe_value = new_value

      # 6. changes辞書に記録
      # 新規なら
      if created:
        # 新しい値だけ記録
        changes[field_name] = new_json_safe_value
      # 更新なら
      else:
        # 前後を記録
        changes[field_name] = {'old': old_json_safe_value, 'new': new_json_safe_value}
```

```
    # 変更辞書を返す
    return changes
```

- `if is_changed:`
- `if ... elif ... else:`
  JSONで　　できる形
  - `isinstance(new_value, (str, int, float, bool, type(None)))`
  - 　　`.isoformat()`
  - 　　`{'id': ..., 'str': ...}`
- `changes[field_name] = ...`
- `changes[field_name] = {'old': ..., 'new': ...}`
- `return changes`

> **この関数が作るもの（イメージ）：**
> `get_changes`
>
> ```
> {'name': {'old': 'admin', 'new': '管理者'}}
> ```
>
> `changes`

18.4 continue（ループの残りを　ばす）

　　`continue`

```
except type(instance).DoesNotExist:
    # この項目は処理せず、次の項目へ
    continue
```

> 語： `continue`　　今回の繰り　しはここで　ち切り、次の繰り　しへ進む
> `break`　　`continue`

18.5 @receiver で自　　（111〜142行　）

> ▼ このブロックがやること:
> `@receiver`

```python
# 保存前に更新前の値をキャッシュ
# member保存の「直前」に呼ぶ登録（§13.3）
@receiver(pre_save, sender=member)
def cache_old_member_instance(sender, instance, **kwargs):
  # 更新前の値を保管（§18.2）
  set_instance_cache(instance)


# 履歴を記録　新規作成、更新（member）
# member保存の「直後」に呼ぶ登録
@receiver(post_save, sender=member)
def log_create_update_member_history(sender, instance, created, **kwargs):
  # 現在のリクエストを取得
  request = get_current_request()
  # 操作者の番号（三項 §8.1）
  session_data = request.session.get('login_No') if request else None

  # 差分計算
  # §18.3で変更点を計算
  changes = get_changes(instance, created)

  # 操作内容判定
  # 新規ならCREATE、更新ならUPDATE（三項 §8.1）
  operation = 'CREATE' if created else 'UPDATE'

  # 履歴記録
  # 履歴レコードを作成
  History.objects.create(
    # 操作種別
    operation=operation,
    # テーブル名
    table_name='member',
    # 対象レコードのid
    record_id=instance.id,
    # 操作した人の番号
    login_No=session_data,
    # 変更内容（§18.3の辞書）
    changes=changes,
  )
```

`@receiver(pre_save, sender=member)` 前

`cache_old_member_instance`

`@receiver(post_save, sender=member)` 後

`log_create_update_member_history`

`get_changes`

`operation = 'CREATE' if created else 'UPDATE'`

`History.objects.create(...)`

> **この仕組みの威 ： `member` 書き手が意識しなくても `@receiver` 全モデル**

> **語： `**kwargs`（クワーグス）**
> `@receiver` `**kwargs`

18.6 削除時の （147〜159行 ）

`@receiver`

```
# 履歴を記録　削除（member）
# member削除の「直後」に呼ぶ登録
@receiver(post_delete, sender=member)
def log_delete_member_history(sender, instance, **kwargs):
  request = get_current_request()
  session_data = request.session.get('login_No') if request else None

  # 履歴記録
  History.objects.create(
    # 操作は削除
    operation='DELETE',
    table_name='member',
    record_id=instance.id,
    login_No=session_data,
    # 削除なので差分は無し（None）
    changes=None,
  )
```

- `@receiver(post_delete, sender=member)`
- `operation='DELETE'`

- `changes=None`

> **と削除のペア:**
> `@receiver`
> **監査ログ（操作の記**
>)

---

19. トラブルシューティング

エラー（種　）	語 の意	よくある原因	処
`IndentationError`			
`SyntaxError`		`:`	`:`
`NameError`			
`TypeError`		`"5" + 5`	`int()` `str()`
`ValueError`		`int("abc")`	
`KeyError`		`[ ]`	`.get()`
`IndexError`		`[10]`	
`AttributeError`			

`member.DoesNotExist`		`.get()`	
`ModuleNotFoundError`			`.` `..` `pip install`

> **エラーの　み　：　　ー　下の行**　`ValueError: invalid literal for int()`

---

20. 演習

ローカル環境　`python`

数と型　`x = "5"`　`y = 5`　`x + x`　`y + y`　`print()`

リストと　　記　`[1, 2, 3, 4, 5, 6]`　`if n % 2 == 0`　`%`　`main_views.py`　`valid_member_numbers`

辞書　`person = {"name": "田中", "no": 12345}`　`person["name"]`　`person.get("age")`　`.get()`

三項演算子

f　字　`name = "admin"`　`count = 3`　`f"..."`

関数　`bigger(a, b)`　`def`

try-except　`int(input())`　`main_utils.py`

else:

tanaka = Person("田中", 30)

_

: ...}}

kosu/ .py

:

} ( )

is None for range

件]

三項演算子 A if 条件 else B f

小 相 : __init__ xtttt__

ァ相ォ相ルは） 相・ models.Model

- try-except import . ..

@